Практична робота №7

Тема: Обробка HTTP-запитів та безпечна передача параметрів.

Мета: Опанувати механізми передачі даних між клієнтом та сервером за допомогою протоколу HTTP. Навчитися розрізняти сфери застосування методів GET та POST, а також впроваджувати базові заходи безпеки (фільтрацію та екранування) для захисту вебзастосунків від поширених вразливостей.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Форми − це контейнер які складаються з елементів вводу інформації користувачем. Форми потрібні для передачі даних від клієнта на сервер. Найчастіше форми використовуються для реєстрації користувачів, заповнення анкет, оформлення замовлення в інтернет магазині, і так далі. Через форми можна відправляти як просту текстову інформацію, так і файли. Велику частину часу програмування на PHP ви будете так чи інакше працювати з формами і даними з них.

HTML описує те, з яких елементів складається форма, і як вона виглядає. Але без приймаючої сторони, тобто сервера, який приймає ці дані і обробляє їх потрібним чином, створювати форми немає ніякого сенсу.

PHP містить безліч засобів для роботи з формами. Це дозволяє дуже просто вирішувати типові завдання, які часто виникають у веб-програмуванні:

- Реєстрація та аутентифікація користувача;
- Відправлення коментарів на форумах і соціальних мережах;
- Оформлення замовлень.

Практично будь-який сучасний сайт містить як мінімум кілька різних HTML-форм.

Відправлення форми

Розглянемо один типовий приклад - форма зворотного зв'язку. Для зв'язку користувачів з авторами сайту, як правило, використовуються форми зворотного зв'язку, де людина вказує ім'я, пошту для зворотного зв'язку і текст свого повідомлення.

Така форма в HTML може виглядати наступним чином:

```
<form name="feedback" method="POST" action="form.php">
  <label>Ваше ім’я: <input type="text" name="name"></label>
  <label>Ваш email: <input type="text" name="email"></label>
  <label>Повідомлення: <textarea name="message"></textarea></label>

  <input type="submit" name="send" value="Відправити">
</form>
```

Це дуже проста форма, що складається з трьох полів і однієї кнопки відправки.

Майже весь наведений код описує зовнішній вигляд і зміст форми, але слід звернути увагу на два атрибути тега `<form>`, які потрібні для вказівки на спосіб обробки даних:

- `method` − цей атрибут використовується для визначення методу HTTP, який буде використаний для передачі даних на сервер. Методом `GET`, яке вказує

серверу просто повернути певний документ. Метод POST повідомляє про намір передати на сервер деяку інформацію, що, втім, не скасовує подальше отримання контенту.

Різниця між GET та POST

- **GET**: Дані передаються через URL. Використовується для отримання ресурсів, пошуку, фільтрації. Дані видимі всім і мають обмеження за довжиною.
- **POST**: Дані передаються в тілі запиту. Використовується для створення/зміни даних (реєстрація, завантаження файлів). Більш безпечно для чутливих даних.

- `action` – містить адресу PHP-скрипта, який повинен обробити цю форму.

Після натискання на кнопку «відправити», браузер виконує POST запит з введеними даними на адресу, вказану в атрибуті `action`.

Обробка форми

Після відправки форми управління передається PHP-скрипту, який повинен отримати передані дані, виконати з ними будь-які дії (наприклад, зберегти в базі даних) і показати результат. Результатом може бути будь-яке повідомлення про успішне завершення операції, наприклад, *«ваші дані успішно відправлено»* .

Тому потрібно в першу чергу навчитися отримувати дані з форми в сценарії. У PHP це робиться легко – **всі дані з форми знаходяться в глобальному асоціативному масиві** `$_POST`. Цей масив завжди буде неявно присутні в сценарії, якщо він був завантажений за методом `POST`. Кожне поле з форми буде знаходитися в масиві, де ключем буде значення атрибута `name`, а значенням вміст поля.

Наприклад, щоб вивести з форми всю інформацію на екран, можна написати такий сценарій:

```
<?php
if (isset($_POST)) {
    print("Ім'я: " . $_POST['name']);
    print("<br>Email: " . $_POST['email']);
    print("<br>Повідомлення: " . $_POST['message']);
}
```

Функція `isset` служить для визначення, чи існує передана їй змінна. Так ми перевіряємо, що сценарій завантажений методом `POST`, тобто була відправлена форма.

Як правило, після обробки форми в PHP, сценарій повинен переадресувати користувача на іншу сторінку. Це пов'язано з тим, що якщо форма була відправлена через метод `POST`, то після оновлення сторінки дані будуть відправлені ще раз, а це, в більшості випадків, небажану поведінку.

Відправка файлів

Крім текстової інформації, існує можливість відправляти на сервер файли будь-яких типів.

Приклад форми для завантаження файлу:

```
  <form name="file_upload" method="POST" action="form.php"
enctype="multipart/form-data">
    <label>Ваш аватар: <input type="file"
name="avatar"></label>
    <input type="submit" name="send" value="Відправити
файл">
  </form>
```

Тут є дві важливі відмінності від першого прикладу:

- Додався новий атрибут `enctype`, який завжди повинен мати значення `multipart/form-data`. Якщо його не буде, то файл не відправиться.
- Сам файл завантажується за допомогою поля з типом «file».

У PHP долучення буде доступний в іншому спеціальному масиві – `$_FILES`.

```
  <?php
  if (isset($_FILES['avatar'])) {
      $file = $_FILES['avatar'];

      print("Завантажено файл с іменем " . $file['name'] .
" та розміром" . $file['size'] . " байт");
  }
```

PHP автоматично зберігає всі завантажені файли в тимчасову папку на сервері. Але зберігати там файли можна, тому що ця директорія періодично очищається, і посилання на такий файл не можна дати на сайті. Рішення тут тільки одне - перемістити завантажений файл в іншу папку. Переміщення файлу завжди виконують відразу після завантаження.

Для початку потрібно переконатися, що в робочій директорії проекту існує папка для зберігання завантажених файлів. Нехай вона називається `uploads`.

Переміщення завантаженого файлу

Для переміщення файлу потрібно знати, де він знаходиться зараз, і адреса папки, в яку він буде переноситися. З поточним адресою все вкрай просто - він вже знаходиться в масиві `$_FILES`. Нова адреса файлу, в свою чергу, складається з шляху до папки і імені файлу. Так як папка `uploads` знаходиться там же, де і поточний сценарій, отримати шлях до неї можна так: `dirname(__FILE__)`.

```
  Код для переміщення файлу в нову папку:
  <?php
  $current_path = $_FILES['avatar']['tmp_name'];
  $filename = $_FILES['avatar']['name'];
  $new_path = dirname(__FILE__) . '/' . $filename;

  move_uploaded_file($current_path, $new_path);
```

Функція `move_uploaded_file()` виконує дві дії:

1. Перевіряє, що файл дійсно завантажений через форму.

2. Переміщує завантажений файл за новою адресою.

Валідація форми

Валідація форми - це перевірка вмісту її полів. Завдання такої перевірки - переконатися, що необхідні поля заповнені, а значення в них відповідають очікуваному формату. Так, наприклад, при реєстрації користувача на сайті, він повинен заповнити поля з адресою електронної пошти та придумати собі пароль. Обидва поля є обов'язковими для заповнення, але значення з поля `email` також має бути коректним email-адресою. Крім текстових значень форми, можна перевіряти формат і розмір файлів.

Загальний підхід до валідації

При виконання валідації будь-якої форми порядок дій буде завжди одним:

1. Сформувати масив з іменами полів, обов'язкових для заповнення.
2. Сформувати масив з правилами для валідації формату полів.
3. Отримати значення всіх полів.

Безпека: Валідація та Санітизація

Ніколи не довіряйте даним від користувача. У PHP 8.1 для цього існують вбудовані фільтри.

Приклад безпечного отримання числа (ID):

```
  // filter_input автоматично перевіряє існування ключа та
його тип
  $id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);

  if ($id === false) {
      die("Помилка: ID має бути цілим числом.");
  }
```

Приклад захисту від XSS (очищення тексту):

Якщо ви виводите текст, отриманий від користувача, обов'язково використовуйте htmlspecialchars(), щоб скрипти <script> не виконалися в браузері.

```
  $username = $_POST['username'] ?? 'Гість';
  echo "Привіт, " . htmlspecialchars($username, ENT_QUOTES,
'UTF-8');
```

1. Фільтри валідації (Validation Filters)

Ці фільтри перевіряють, чи відповідають дані правилам. Якщо дані некоректні, функція повертає false.

Фільтр	Опис	Приклад використання
`FILTER_VALIDATE_EMAIL`	Перевіряє коректність e-mail адреси.	`filter_var($email, FILTER_VALIDATE_EMAIL)`
`FILTER_VALIDATE_INT`	Перевіряє, чи є значення цілим числом.	Дозволяє задати `min_range` та `max_range`.

FILTER_VALIDATE_FLOAT	Перевіряє, чи є значення числом з плаваючою крапкою.	Розуміє і `.` і `,` (при налаштуванні).
FILTER_VALIDATE_IP	Перевіряє IP-адресу (IPv4 або IPv6).	Можна прапорцями заборонити приватні IP.
FILTER_VALIDATE_URL	Перевіряє правильність формату посилання.	Обов'язкова наявність протоколу (http/https).
FILTER_VALIDATE_BOOL	Повертає `true` для "1", "true", "on", "yes".	Зручно для чекбоксів у формах.

2. Фільтри очищення (Sanitization Filters)

Ці фільтри видаляють небажані символи зі зміною самого значення. Вони завжди повертають рядок.

- **FILTER_SANITIZE_NUMBER_INT**: Видаляє всі символи, крім цифр, знаків плюс та мінус.

  Приклад: рядок `"100$"` перетвориться на `"100"`.
- **FILTER_SANITIZE_EMAIL**: Видаляє всі символи, крім латинських літер, цифр та спеціальних символів, дозволених в email.
- **FILTER_SANITIZE_URL**: Видаляє символи, які не можуть бути в посиланні.
- **FILTER_SANITIZE_SPECIAL_CHARS**: Еквівалент `htmlspecialchars`. Замінює <, >, `&`, `"` на HTML-сутності.

Обробка форми в одному файлі PHP

У попередніх випадках форма була розміщена на одній сторінці, а відправлялася на іншу. Насправді це не обов'язково. Якщо залишити атрибут `action` порожнім або прибрати його зовсім, то форма буде відправлятися на цю ж сторінку.

Як це буде працювати: при першому заході на сторінку ми заповнимо форму даними і натиснемо на кнопку. Після цього сторінка оновиться і її код виконається знову, але вже з даними форми.

Давайте подивимося на прикладі. Нехай у нас в одному файлі є форма і її обробка:

```
<form action="" method="GET">
   <input name="test1">
   <input name="test2">
   <input type="submit">
</form>

<?php
   var_dump($_GET);
?>
```

При першому заході на сторінку функція `var_dump` виведе порожній масив. А після відправки форми він виведе вже дані форми. Тобто перший раз `$_GET` буде порожній, а другий раз – буде містити дані форми.

Це може привести до проблем. Нехай, наприклад, ми в форму будемо вводити числа і хочемо вивести суму цих чисел на екран:

```
<form action="" method="GET">
   <input name="test1">
   <input name="test2">
   <input type="submit">
</form>

<?php
   echo $_GET['test1'] + $_GET['test2'];
?>
```

В цьому випадку при першому заході на сторінку ми побачимо помилки PHP, пов'язані з тим, що масив $_GET порожній, а ми звертаємося до її елементів.

Тут слід сказати, що у вас помилки можуть і не з'явиться в браузері. В цьому випадку перевірте, що у вас включений висновок помилок PHP, а також переконайтеся, що у вас перший візит на сторінку і в адресному рядку немає даних форми.

Давайте виправимо проблему. Для цього додамо умова, в якому будемо перевіряти те, що форма була відправлена.

Наприклад, можна перевіряти $_GET на «Не порожні» значення. Якщо $_GET не порожній то форма була відправлена і можна виконувати підсумовування. В іншому випадку у нас ще перший візит на сторінку і підсумовування виконано не буде. Отже, ось виправлений код:

```
<form action="" method="GET">
   <input name="test1">
   <input name="test2">
   <input type="submit">
</form>

<?php
   if (!empty($_GET)) {
       echo $_GET['test1'] + $_GET['test2'];
   }
?>
```

ЗАВДАННЯ ДЛЯ САМОСТІЙНОГО ВИКОНАННЯ

Завдання 1. **Розумний калькулятор (GET)**

Створіть сторінку calc.php, яка приймає три параметри через URL: a, b (числа) та op (операція: add, sub, mult, div).

- *Вимога:* Використати filter_input для перевірки, що a та b є числами.
- *Безпека:* Реалізувати перевірку на ділення на нуль.
- *PHP 8.1:* Використати оператор match для вибору операції.

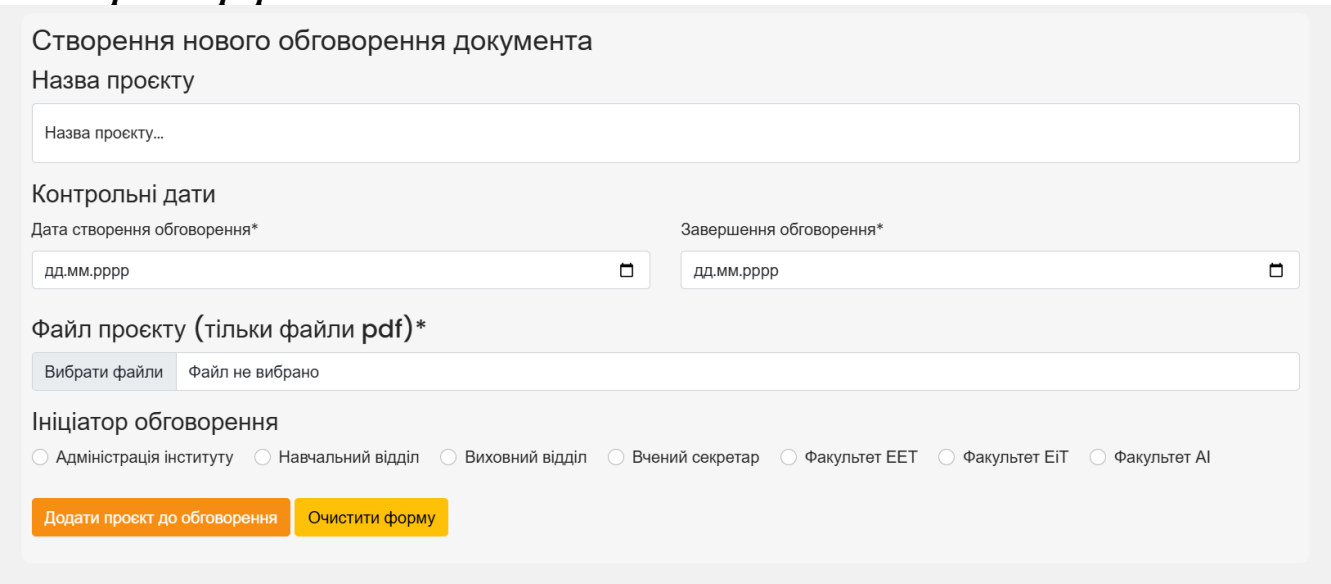
***Завдання 2.* Система документообігу**

Необхідно реалізувати серверний сценарій `create_discussion.php` для обробки форми створення нового обговорення документа. Форма передає дані методом **POST**.

Поля форми для обробки:

1. **Назва проєкту** (текстове поле).
2. **Дата створення** та **Дата завершення** (поля типу date).
3. **Файл проєкту** (завантаження PDF).
4. **Ініціатор обговорення** (набір радіокнопок: Адміністрація, Навчальний відділ, Виховний відділ тощо).

Алгоритм виконання (Завдання):

1. *Прийом та санітизація тексту:*
   - Отримайте "Назву проєкту". Використовуйте `filter_input` з фільтром `FILTER_SANITIZE_SPECIAL_CHARS` для захисту від XSS.
2. *Валідація дат:*
   - Перевірте, чи були надіслані обидві дати.
   - ***Логічна перевірка:*** Дата завершення не може бути раніше дати створення.
   - *Підказка:* Використовуйте об'єкт `DateTime` для порівняння.
3. *Обробка файлу (PDF):*
   - Перевірте, чи файл було завантажено без помилок (`$_FILES['project_file']['error'] === 0`).
   - Перевірте тип файлу. Дозволяється лише `application/pdf`.
4. *Валідація вибору ініціатора:*
   - Отримайте значення вибраної радіокнопки.
   - Створіть масив дозволених значень (білий список): `['admin', 'study', 'edu', 'secretary', 'eet', 'eit', 'ai']`.
   - Перевірте, чи надіслане значення входить у цей масив за допомогою `in_array()`.
5. *Фінальний вивід:*
   - Якщо всі дані валідні –вивести зведену таблицю з даними проєкту.
   - Якщо є помилки – вивести список зауважень червоним кольором.

Зразок форми